WINDOWS SERVICE & PROCESS MONITORING AGENT

Practical Work Documentation

Student Name:

Vaishali Vasant Kadam

Project Title:

Windows Service & Process Monitoring Agent

Submitted For:

Internship Practical Documentation

GitHub:

https://github.com/kadamvaishali378/Windows-Service-and-Process-Monitoring-Agent

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my internship mentor and the organization for providing me the opportunity to work on the project **“Windows Service & Process Monitoring Agent.”**

This project allowed me to gain practical exposure to system-level monitoring and defensive cybersecurity practices. Through this work, I learned how Windows processes and services operate internally and how suspicious behavior can be detected using behavioral monitoring techniques.

I would also like to thank my team members and colleagues for their continuous support, valuable feedback, and guidance throughout the internship. Their assistance helped me overcome several technical challenges and complete the project successfully.

Finally, I would like to express my gratitude to my instructors and institution for providing the foundational cybersecurity knowledge that made this project possible.

DECLARATION

I hereby declare that the project titled **“Windows Service & Process Monitoring Agent”** submitted by me is an original work carried out during my internship as part of the requirements for my academic program in Cybersecurity / Information Security.

I affirm that this project has been executed by me independently, under the guidance and supervision of my internship mentor. The work presented in this report has not been submitted previously for any other academic or professional qualification.

All sources of information, references, and external materials used in the preparation of this report have been duly acknowledged in the appropriate sections. I have ensured that any code, diagrams, data, or concepts that are not my original work have been properly cited and credited.

I also acknowledge that the insights, learning, and outcomes described in this report are based on my own practical experience gained during the internship and accurately reflect the scope and results of the project.

I take full responsibility for the authenticity, accuracy, and integrity of this project report.

Student Name: Vaishali Vasant Kadam

Contents

ACKNOWLEDGEMENT 2
DECLARATION 3
1. INTRODUCTION 6
2. PROJECT OVERVIEW / DESCRIPTION 7
3. PROBLEM STATEMENT 8
4. PROJECT OBJECTIVES 9
5. SCOPE OF THE PROJECT 10
6. TOOLS & TECHNOLOGIES USED 11
7. SYSTEM ARCHITECTURE 13
8. WORKFLOW / FLOWCHART 15
9. IMPLEMENTATION / CODE DESCRIPTION 18
10. DETECTION TECHNIQUES IMPLEMENTED 26
11. TESTING METHODOLOGY 27
12. SAMPLE OUTPUTS / RESULTS 28
13. OBSERVATIONS / ANALYSIS 29
14. ADVANTAGES OF THE SYSTEM 30
15. LIMITATIONS / DISADVANTAGES 31
16. FUTURE SCOPE 32
17. LEARNING OUTCOMES 33
18. CONCLUSION 34
19. PROJECT DELIVERABLES 35
20. REFERENCES / BIBLIOGRAPHY 36

Table of Figures

Figure 1: System Architecture 14
Figure 2: Flowchart 17
Figure 3: main.py 19
Figure 4: Terminal Output 20
Figure 5: alerts.log 21
Figure 6: Suspicious alerts 22
Figure 7: Total Services Found 23
Figure 8: Final Report.txt 24

1. INTRODUCTION

Modern organizations depend heavily on **Windows operating systems** for everyday operations. Their widespread adoption makes them a frequent target for cyber attacks. Attackers often exploit system processes, background services, and startup mechanisms to maintain persistence or execute malicious commands while remaining undetected.

Traditional antivirus solutions rely largely on known malware signatures, which may fail to detect **advanced or emerging threats**. To address this limitation, behavior-based monitoring techniques have become essential for detecting suspicious activity.

This project focuses on the design and implementation of a **Windows Service & Process Monitoring Agent**. The agent continuously monitors running processes and Windows services, analyzing **parent–child relationships, execution paths, and service configurations** to identify anomalies that could indicate malicious activity.

Through this project, I gained **practical exposure to defensive cybersecurity techniques**, similar to those used in real-world Security Operations Centers (SOC), providing hands-on experience in threat detection and endpoint monitoring.

2. PROJECT OVERVIEW / DESCRIPTION

The **Windows Service & Process Monitoring Agent** is a Python-based monitoring system designed to analyze **Windows processes and services** to detect abnormal or suspicious activity.

The agent continuously monitors system behavior and performs several security checks, including:

- **Process Monitoring:** Tracks active processes and their parent–child relationships to detect unusual execution chains.
- **Execution Path Analysis:** Flags processes running from suspicious or user-writable directories (e.g., Temp, Downloads, AppData).
- **Unauthorized Process Detection:** Identifies unknown or untrusted processes not present in a baseline whitelist.
- **Service Auditing:** Inspects Windows services for abnormal configurations, unusual startup paths, or newly registered services.
- **Alerting and Reporting:** Generates structured logs and reports summarizing detected anomalies and suspicious behaviors.

The system follows a **modular architecture**, where each component focuses on a specific monitoring task. The collected data is analyzed using **rule-based detection logic** to identify patterns typically associated with malware or unauthorized activity.

This project simulates the type of **endpoint monitoring performed in enterprise Security Operations Centers (SOC)**, providing hands-on experience in practical cybersecurity monitoring and defensive techniques.

3. PROBLEM STATEMENT

Modern cyber attacks have become increasingly sophisticated, often avoiding detection by traditional signature-based antivirus systems. Instead of relying on obvious malware files, attackers exploit legitimate system components such as Windows processes, background services, and startup mechanisms. These techniques allow malicious activity to blend into normal system behavior, making detection difficult.

Common techniques used by attackers include:

- **Creating malicious Windows services for persistence:** Attackers often register services that automatically start with the system, ensuring that their malicious code runs continuously, even after system reboots.
- **Executing commands through trusted applications:** Malicious actors may abuse legitimate applications, such as Microsoft Word, Excel, or browsers, to execute scripts or commands that appear benign.
- **Running processes from temporary or user-writable directories:** By executing from non-standard paths, malware can evade security controls and avoid raising suspicion.
- **Launching command shells or scripting engines via document editors or browsers:** This method allows attackers to gain control of the system while hiding within trusted parent processes.

These attack strategies exploit the normal behavior of Windows operating systems and make it difficult for traditional antivirus solutions, which rely on known malware signatures, to detect new or sophisticated threats.
There is a clear need for a **behavior-based monitoring solution** that observes runtime system activity, identifies anomalies, and detects suspicious behavior in real time. Instead of relying solely on pre-existing signatures, the solution should analyze process hierarchies, execution paths, and service configurations to identify activities that deviate from normal patterns.

4. PROJECT OBJECTIVES

The **Windows Service & Process Monitoring Agent** was developed to enhance endpoint security on Windows systems by providing real-time monitoring and detection of suspicious activity. The project objectives are outlined as follows:

1. Monitor Running Windows Processes
The system continuously tracks all active processes on the host machine. It collects essential details such as process name, Process ID (PID), Parent Process ID (PPID), and executable path. Monitoring these attributes allows the agent to understand normal system behavior, reconstruct process trees, and identify anomalies that may indicate malicious activity. This continuous observation is crucial for detecting unauthorized executions or unusual process behavior.

2. Detect Suspicious Parent–Child Process Relationships
Malware often hides by leveraging legitimate applications as parent processes—for example, using Microsoft Word to launch a command shell. The agent analyzes parent–child relationships to detect such abnormal execution chains. Identifying unexpected process hierarchies helps flag potential attacks early, including macro-based exploits, script injections, or automated tools misused by attackers.

3. Audit Windows Services
Windows services often run with elevated privileges and can be exploited for persistence. The agent audits services to identify unusual startup configurations, services running from suspicious file paths, or newly registered services that may indicate backdoors. By inspecting these service configurations, the system helps prevent long-term unauthorized access and monitors for indicators of compromise.

4. Detect Unauthorized or Unknown Processes
To minimize false positives and focus on real threats, the agent maintains a baseline of trusted processes. Any process not present in this baseline is flagged as potentially unauthorized. This allows detection of unknown or rogue processes, including malware, scripts, or user-installed software running without authorization. It strengthens system integrity by alerting administrators to unusual activity.

5. Generate Structured Alerts and Reports
The agent produces clear, structured alerts whenever suspicious behavior is detected. Alerts include details such as the type of anomaly, severity level, process/service information, and timestamp. Additionally, it generates comprehensive security reports that summarize all detected anomalies. These reports are suitable for documentation, auditing, or further analysis in a Security Operations Center (SOC) environment.

Overall Objective:
The combined aim of these objectives is to provide a **behavior-based endpoint monitoring solution** that goes beyond traditional antivirus detection. By observing runtime behavior, analyzing system processes, and auditing services, the project ensures early identification of malicious activity, persistence attempts, and unauthorized system changes.

5. SCOPE OF THE PROJECT

The **Windows Service & Process Monitoring Agent** focuses on monitoring and analyzing system activity at the **endpoint level** on Windows operating systems. The project aims to detect suspicious behaviors related to processes and services that may indicate potential security threats.

The system is designed to provide **basic security monitoring capabilities** by analyzing process behavior, execution paths, and Windows service configurations. It helps identify abnormal activities that may be associated with malware execution, unauthorized programs, or persistence mechanisms.

Key Features of the System

• **Real-Time Monitoring of Running Processes**
The monitoring agent continuously observes active processes running on the system and collects information such as process name, process ID, and executable path.

• **Parent–Child Process Relationship Analysis**
The system analyzes how processes are created and identifies unusual parent–child relationships that may indicate suspicious behavior.

• **Detection of Suspicious Execution Paths**
Processes running from unusual directories such as temporary folders or user-writable locations are flagged as potentially suspicious.

• **Auditing of Windows Services**
The system audits Windows services to identify suspicious services, unusual startup configurations, or services running from abnormal locations.

• **Detection of Unauthorized Processes Using Baseline Comparison**
A trusted baseline list of normal processes is created and used to compare against currently running processes. Any unknown or unauthorized process can be flagged for further investigation.

• **Alert Generation and Reporting**
When suspicious activity is detected, the system generates alerts and produces structured reports summarizing the monitoring results.

6. TOOLS & TECHNOLOGIES USED

The **Windows Service & Process Monitoring Agent** was developed using several programming tools, libraries, and system utilities that support process monitoring and system analysis.

1. Programming Language

• Python 3.x
Python was used as the primary programming language for developing the monitoring agent. It provides powerful libraries and easy system interaction capabilities, making it suitable for building security monitoring tools.

2. Libraries / Modules

• psutil – Process Monitoring
The *psutil* library is used to collect information about running processes. It helps retrieve details such as process name, process ID (PID), parent process ID (PPID), and executable path.

• wmi – Windows Service Enumeration
The *WMI (Windows Management Instrumentation)* library is used to access system-level information about Windows services, including service name, status, and startup configuration.

• win32service – Windows Service Management
This module allows interaction with Windows services and helps retrieve service configuration details for auditing purposes.

• datetime – Timestamp Generation
The *datetime* module is used to generate timestamps for alerts and logs so that security events can be recorded with accurate time information.

• os – File Path Handling
The *os* module is used to manage file paths and directories while analyzing the location from which processes are executed.

3. Development Tools

• Visual Studio Code / PyCharm
These Integrated Development Environments (IDEs) were used for writing, editing, and debugging the Python scripts used in the monitoring system.

4. Operating System

• **Windows 10 / Windows 11**
The project was designed and tested on Windows operating systems because it focuses on monitoring Windows processes and services.

5. Documentation Tools

• **Microsoft Word** – Used for writing the project documentation.
• **Draw.io** – Used for creating system architecture diagrams and workflow flowcharts.

7. SYSTEM ARCHITECTURE

The **Windows Service & Process Monitoring Agent** follows a **modular architecture**, where each module is responsible for a specific monitoring task. This design improves maintainability, scalability, and clarity, ensuring that system data collection, analysis, anomaly detection, and reporting work seamlessly together.

1. Process Monitoring Module
This module continuously collects information about all active processes on the system. It records key attributes such as process name, Process ID (PID), Parent Process ID (PPID), and executable path. The gathered data is then forwarded to the behavior analysis engine for detailed inspection. This module forms the foundation for detecting suspicious process activity.

2. Behavior Analysis Engine
The behavior analysis engine examines relationships between processes and evaluates their execution patterns. By analyzing parent–child process chains, the engine can detect unusual behavior, such as trusted applications launching unexpected command-line tools or scripting engines. This module is critical for identifying behavioral anomalies indicative of malware.

3. Baseline Comparison Module
This module maintains a list of trusted processes as a **baseline**. Any currently running process not present in the baseline is flagged as unknown or unauthorized. Comparing runtime processes against the baseline helps minimize false positives while effectively detecting potentially malicious programs.

4. Service Audit Module
The service audit module enumerates all Windows services and collects details such as service name, startup type, current status, and executable path. By analyzing this information, the system can identify suspicious services, abnormal startup configurations, or newly registered services that may indicate malware persistence.

5. Alert Logging Module
When the system detects suspicious activity, the alert logging module records the event with all relevant details, including timestamp, process/service name, type of alert, and severity level. These logs provide a historical record of system events and support further forensic analysis.

6. Report Generation Module
This module compiles all detected alerts and monitoring results into a **structured report**. Reports summarize system activity, highlight detected anomalies, and classify alerts by severity. These reports are useful for documentation, auditing, or use within a Security Operations Center (SOC).

Overall Architecture:
The modular approach allows the monitoring agent to efficiently detect suspicious behavior at the process and service level while generating actionable logs and reports. Each module complements the others, providing a comprehensive endpoint monitoring solution that mirrors enterprise security practices.

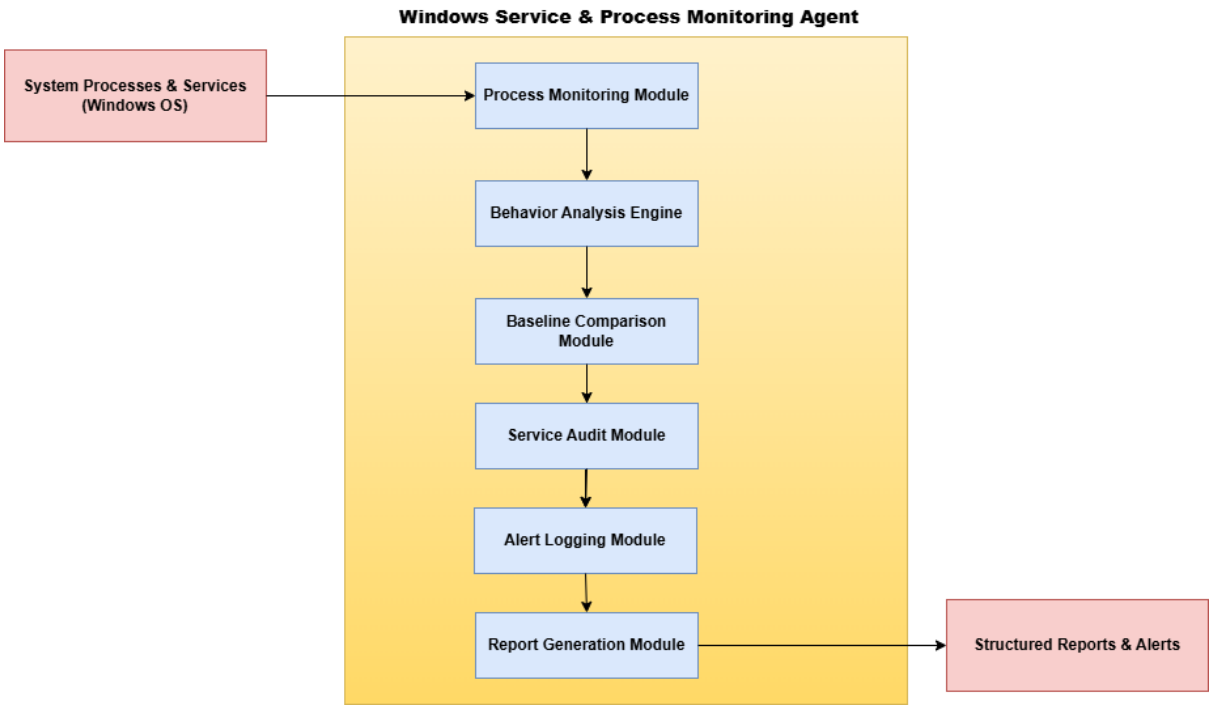


Figure 1: System Architecture

8. WORKFLOW / FLOWCHART

The The **Windows Service & Process Monitoring Agent** follows a structured and sequential workflow to continuously monitor system activity, analyze processes and services, detect anomalies, and generate actionable reports. The workflow ensures that all critical steps of process and service monitoring are systematically performed to detect suspicious behavior effectively.

Step 1 – Enumerate Running Processes
The system begins by collecting information about all active processes running on the Windows system. Using Python libraries like psutil, the agent gathers critical details including:

- Process Name
- Process ID (PID)
- Parent Process ID (PPID)
- Executable Path

This step forms the foundation for detecting unusual process behavior or unauthorized programs.

Step 2 – Analyze Parent–Child Relationships
The collected process data is then analyzed to examine the parent–child relationships between processes. By observing which process spawned which, the system can identify abnormal execution chains, such as a trusted application unexpectedly launching a command shell or scripting engine. This analysis helps detect common malware techniques like process injection or living-off-the-land attacks.

Step 3 – Audit Windows Services
The agent inspects all Windows services to collect information such as:

- Service Name
- Startup Type (Automatic, Manual, Disabled)
- Current Status (Running/Stopped)
- Executable Path

Suspicious services, like those with unusual startup configurations or unknown executables, are flagged as potential persistence mechanisms used by malware.

Step 4 – Compare Processes With Baseline
The system compares currently running processes with a previously generated trusted baseline of known processes. Any process not included in this baseline may be considered unknown or unauthorized. This comparison helps reduce false positives and focuses on detecting genuinely suspicious activity.

Step 5 – Detect Suspicious Behavior
Using **predefined detection rules**, the system analyzes process relationships, execution paths, and service configurations to identify suspicious patterns. Examples include:

- Processes running from temporary directories
- Unexpected parent-child process chains
- Newly registered services from unusual paths

Step 6 – Generate Alerts
Whenever suspicious activity is detected, the monitoring agent generates detailed alerts. Each alert contains:

- Process/Service Name
- Parent Process (if applicable)
- Severity Level (High, Medium, Low)
- Timestamp

These alerts provide actionable insights for security monitoring and investigation.

Step 7 – Generate Final Report
Finally, the system compiles all alerts and monitoring results into a structured report. The report summarizes:

- Total Processes Analyzed
- Total Alerts Generated
- Severity Breakdown
- Notable Suspicious Activity

This report serves as documentation and can support auditing or further analysis in a SOC environment.

Workflow Representation:

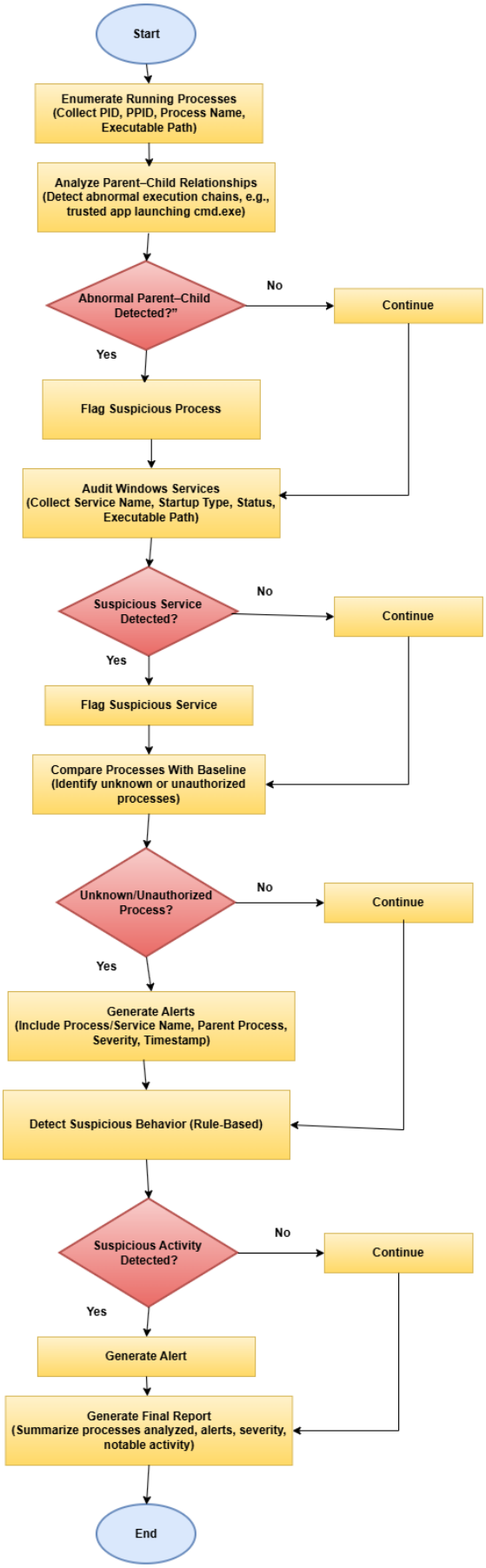


Figure 2: Flowchart

9. IMPLEMENTATION / CODE DESCRIPTION

The **Windows Service & Process Monitoring Agent** is implemented using multiple Python scripts, where each script performs a specific function within the monitoring system. This modular design helps keep the system organized, easier to maintain, and scalable for future improvements.

1. main.py
This script acts as the main controller of the monitoring system. It initializes the monitoring process and coordinates the execution of different modules. The script runs the monitoring cycle at regular intervals, collects results from other modules, and triggers the report generation process.

2. process_monitor.py
This module is responsible for collecting information about all running processes on the system. Using the *psutil* library, it retrieves important details such as the **process name, process ID (PID), parent process ID (PPID), and executable path**. This information is used for further analysis by the detection module.

3. detector.py
This module implements the core **detection logic** of the system. It analyzes process relationships, execution paths, and compares running processes with the trusted baseline. The module identifies suspicious parent–child process relationships and unauthorized processes that may indicate malicious activity.

4. service_audit.py
This script collects information about Windows services using the **WMI library**. It extracts service details such as service name, startup type, running status, and executable path. The data is analyzed to detect suspicious services that may be used for persistence.

5. baseline_generator.py
This module generates a **baseline list of trusted processes** that normally run on the system. During monitoring, the system compares currently running processes against this baseline to detect unknown or unauthorized processes.

6. report_generator.py
This module generates structured monitoring reports based on the alerts detected during the monitoring cycle. The report includes details such as the number of processes analyzed, alerts generated, and severity levels.

Example Detection Logic

Suspicious Parent–Child Detection

```
if (parent_name, child_name) in suspicious_pairs:
    alerts.append({
        "severity": "HIGH",
        "type": "Suspicious Parent-Child",
        "parent": parent_name,
```

```
        "child": child_name
    })
```

Unauthorized Process Detection

```
if process_name not in whitelist:
    alerts.append({
        "severity": "HIGH",
        "type": "Unauthorized Process",
        "process": process_name
    })
```

These detection rules allow the monitoring agent to identify suspicious behaviors and generate alerts for further analysis.

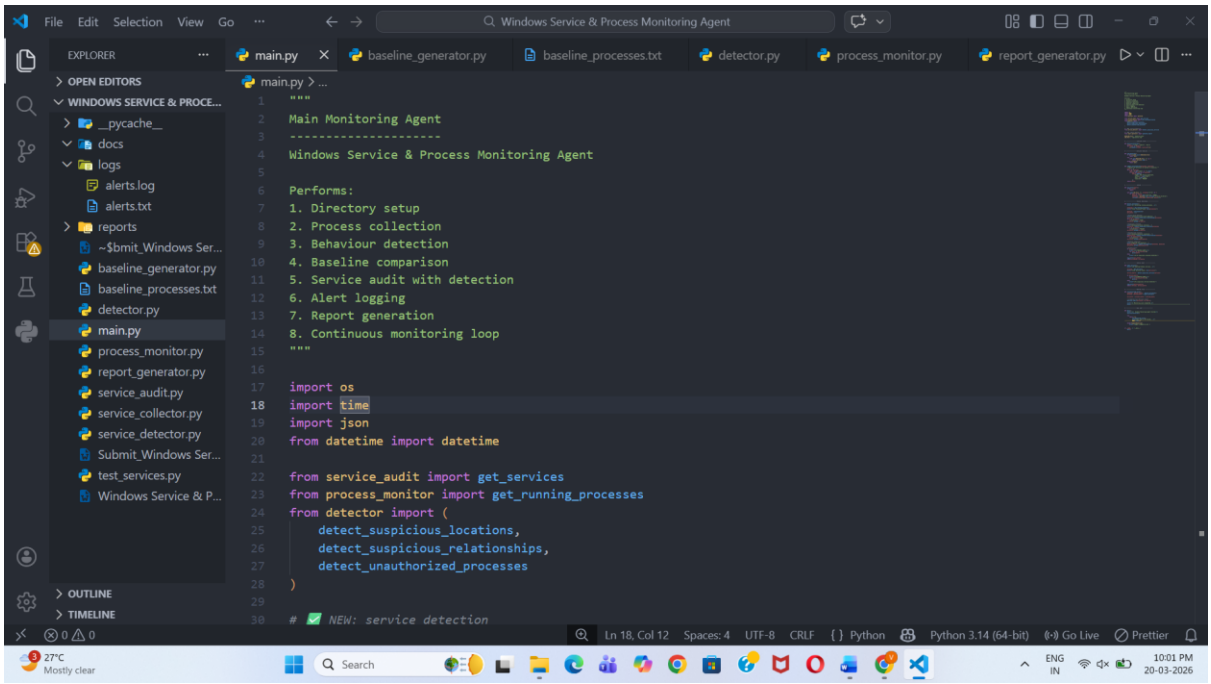


Figure 3: main.py

```
Command Prompt - python r

Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Vaishali>cd C:\Users\Vaishali\Desktop\Windows ServiceProcess Monitoring Agent

C:\Users\Vaishali\Desktop\Windows ServiceProcess Monitoring Agent>python main.py

🛡️ Windows Monitoring Agent Started


🔍 Collecting running processes...

Total Processes Found: 275

Analyzing parent-child behaviour...
Checking execution locations...
Checking whitelist violations...
Comparing with baseline...

🚨 PROCESS ALERTS:

{'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 7600}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 9948}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'smartscreen.exe', 'pid': 3204, 'path': 'C:\\Windows\\Sys
tem32\\smartscreen.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'MicrosoftStartFeedProvider.exe', 'pid': 3688, 'path': 'C
:\\Program Files\\WindowsApps\\Microsoft.StartExperiencesApp_1.241.0.0_x64__8wekyb3d8bbwe\\MicrosoftStartFeedProvider\\MicrosoftStartFeedProvider.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 6348, 'path': 'C:\\Windo
ws\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 8532, 'path': 'C:\\Windo
ws\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'ShellHost.exe', 'pid': 8928, 'path': 'C:\\Windows\\Syste
m32\\ShellHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CrossDeviceResume.exe', 'pid': 9272, 'path': 'C:\\Window
s\\SystemApps\\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\\CrossDeviceResume.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'WidgetBoard.exe', 'pid': 11156, 'path': 'C:\\Program Fil
es\\WindowsApps\\MicrosoftWindows.Client.WebExperience_526.1202.40.0_x64__cw5n1h2txyewy\\WidgetBoard.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'SDXHelper.exe', 'pid': 12552, 'path': 'C:\\Program Files
\\Microsoft Office\\root\\Office16\\SDXHelper.exe'}

🔧 Auditing Windows services...
```

Figure 4: Terminal Output

```
Command Prompt

{'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 7600}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 9948}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'smartscreen.exe', 'pid': 3204, 'path': 'C:\\Windows\\Sys
tem32\\smartscreen.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'MicrosoftStartFeedProvider.exe', 'pid': 3688, 'path': 'C
:\\Program Files\\WindowsApps\\Microsoft.StartExperiencesApp_1.241.0.0_x64__8wekyb3d8bbwe\\MicrosoftStartFeedProvider\\MicrosoftStartFeedProvider.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 6348, 'path': 'C:\\Windo
ws\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 8532, 'path': 'C:\\Windo
ws\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'ShellHost.exe', 'pid': 8928, 'path': 'C:\\Windows\\Syste
m32\\ShellHost.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CrossDeviceResume.exe', 'pid': 9272, 'path': 'C:\\Window
s\\SystemApps\\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\\CrossDeviceResume.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'WidgetBoard.exe', 'pid': 11156, 'path': 'C:\\Program Fil
es\\WindowsApps\\MicrosoftWindows.Client.WebExperience_526.1202.40.0_x64__cw5n1h2txyewy\\WidgetBoard.exe'}
{'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'SDXHelper.exe', 'pid': 12552, 'path': 'C:\\Program Files
\\Microsoft Office\\root\\Office16\\SDXHelper.exe'}

🔧 Auditing Windows services...

Total Services Found: 291

🚨 SERVICE ALERTS:

{'type': 'Service missing executable path', 'service': 'LSM', 'severity': 'MEDIUM'}
{'type': 'Service missing executable path', 'service': 'NetSetupSvc', 'severity': 'MEDIUM'}
{'type': 'Service running from suspicious location', 'service': 'BcastDVRUserService_c3c61', 'path': 'C:\\WINDOWS\\system32\\svchost.exe -k BcastDVRUserServ
ice', 'severity': 'HIGH'}
📄 Security report generated: reports/final_report.txt

✅Monitoring cycle completed.

--------------------------------------------------------------------------------
Next scan in 60 seconds...


Agent stopped by user.

C:\Users\Vaishali\Desktop\Windows ServiceProcess Monitoring Agent>
```

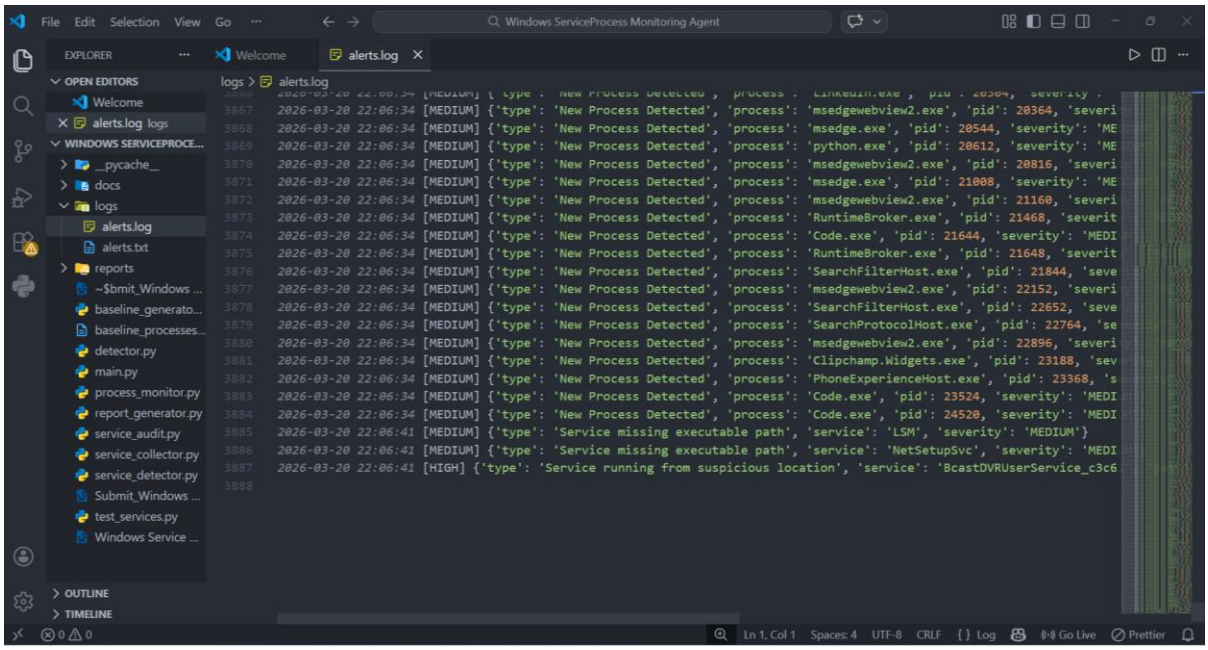
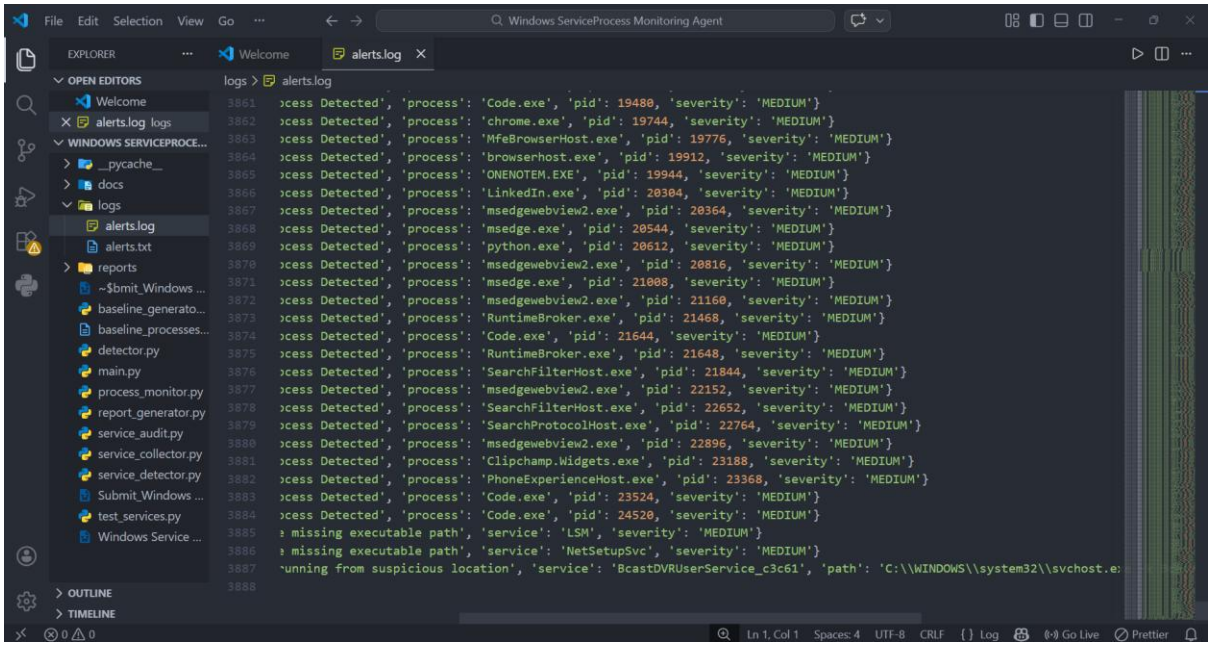


Figure 5: alerts.log



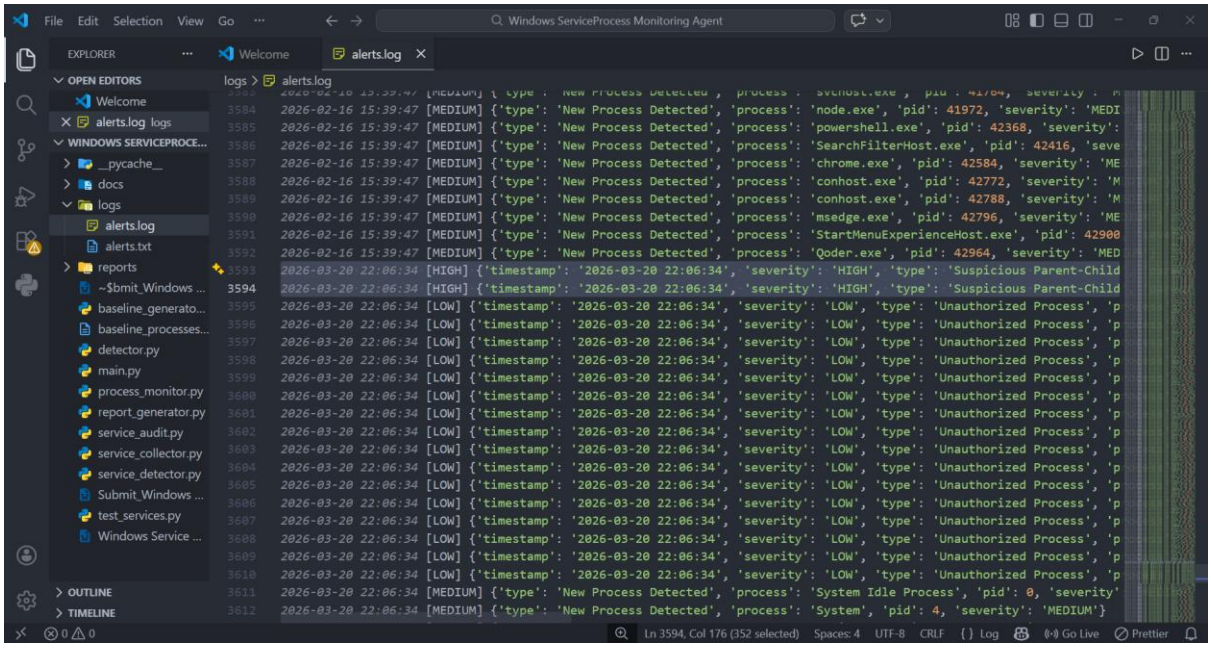
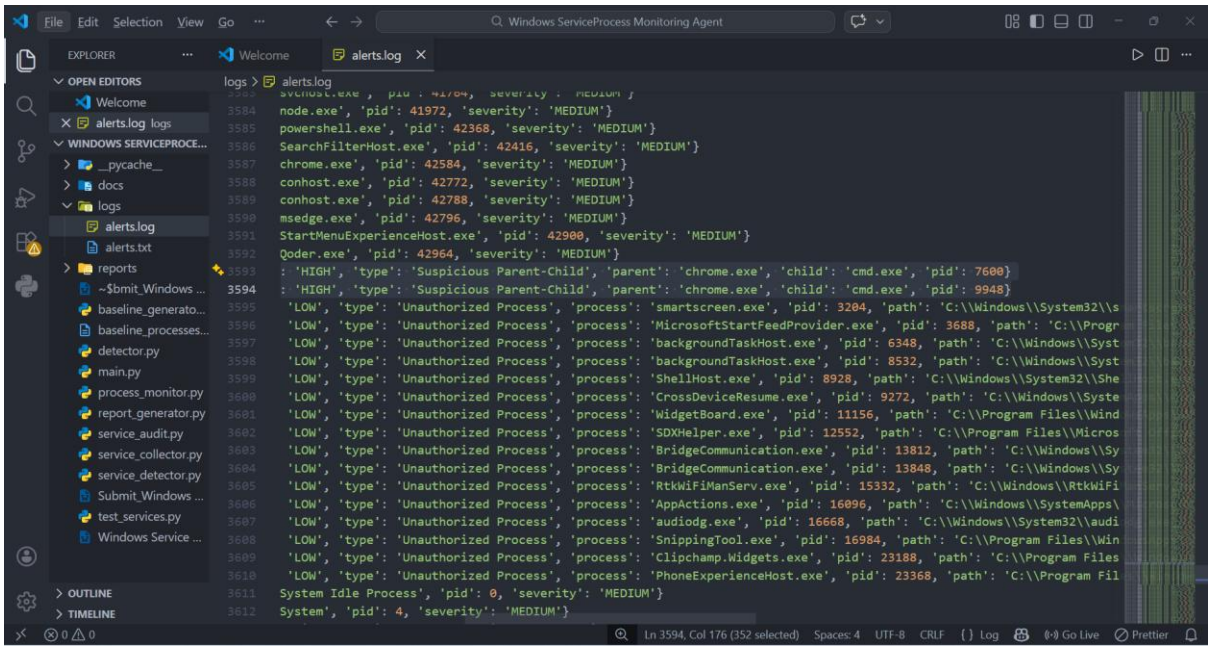
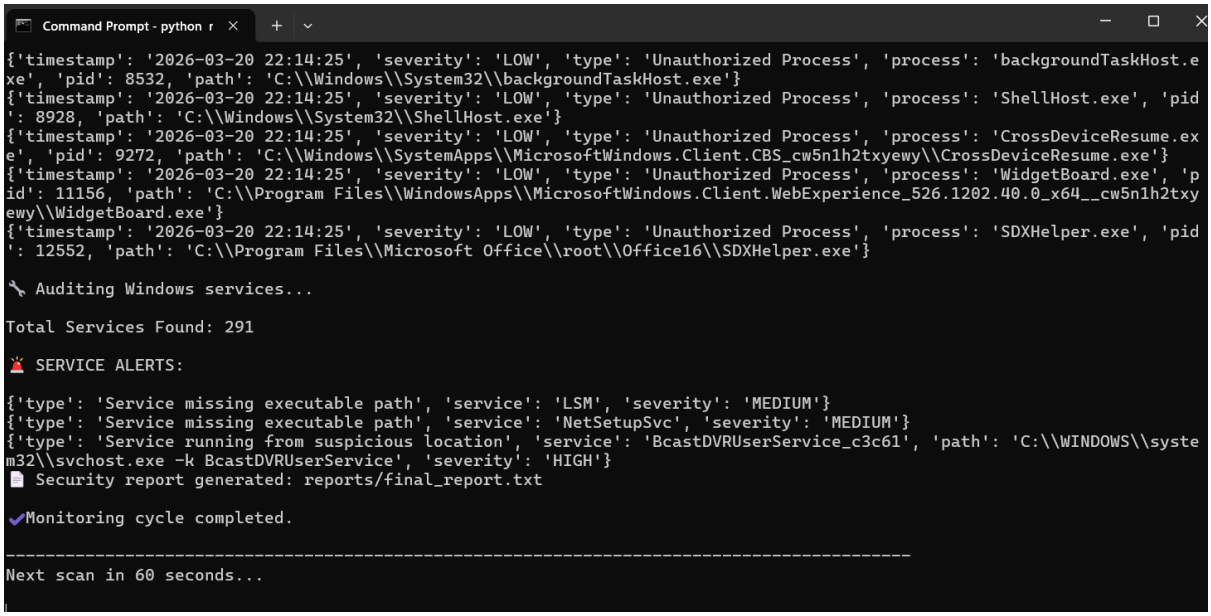


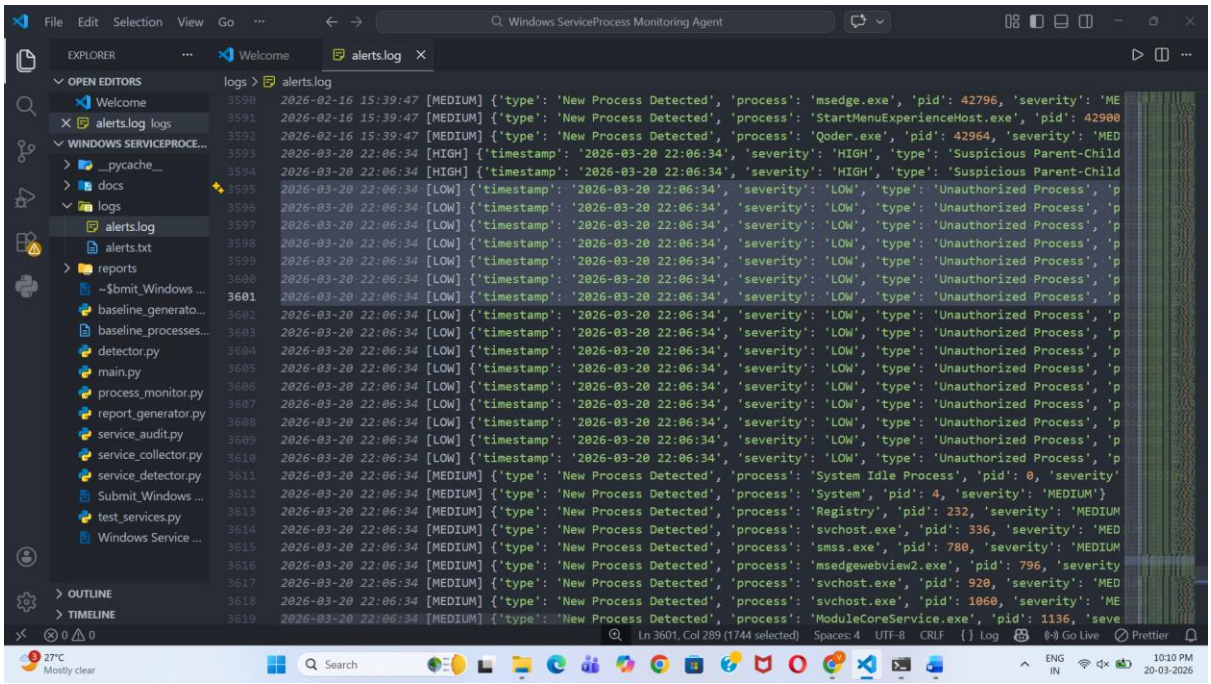
Figure 6: Suspicious alerts





```
Command Prompt - python r
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.e
xe', 'pid': 8532, 'path': 'C:\\Windows\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'ShellHost.exe', 'pid
': 8928, 'path': 'C:\\Windows\\System32\\ShellHost.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CrossDeviceResume.ex
e', 'pid': 9272, 'path': 'C:\\Windows\\SystemApps\\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\\CrossDeviceResume.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'WidgetBoard.exe', 'p
id': 11156, 'path': 'C:\\Program Files\\WindowsApps\\MicrosoftWindows.Client.WebExperience_526.1202.40.0_x64__cw5n1h2txy
ewy\\WidgetBoard.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'SDXHelper.exe', 'pid
': 12552, 'path': 'C:\\Program Files\\Microsoft Office\\root\\Office16\\SDXHelper.exe'}

🔧 Auditing Windows services...

Total Services Found: 291

🚨 SERVICE ALERTS:

{'type': 'Service missing executable path', 'service': 'LSM', 'severity': 'MEDIUM'}
{'type': 'Service missing executable path', 'service': 'NetSetupSvc', 'severity': 'MEDIUM'}
{'type': 'Service running from suspicious location', 'service': 'BcastDVRUserService_c3c61', 'path': 'C:\\WINDOWS\\syste
m32\\svchost.exe -k BcastDVRUserService', 'severity': 'HIGH'}
📄 Security report generated: reports/final_report.txt

✔Monitoring cycle completed.

--------------------------------------------------------------------------------
Next scan in 60 seconds...
```

Figure 7: Total Services Found



```
2026-02-16 15:39:47 [MEDIUM] {'type': 'New Process Detected', 'process': 'msedge.exe', 'pid': 42796, 'severity': 'ME
2026-02-16 15:39:47 [MEDIUM] {'type': 'New Process Detected', 'process': 'StartMenuExperienceHost.exe', 'pid': 42900
2026-02-16 15:39:47 [MEDIUM] {'type': 'New Process Detected', 'process': 'Qoder.exe', 'pid': 42964, 'severity': 'MED
2026-03-20 22:06:34 [HIGH] {'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child
2026-03-20 22:06:34 [HIGH] {'timestamp': '2026-03-20 22:06:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [LOW] {'timestamp': '2026-03-20 22:06:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'p
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'System Idle Process', 'pid': 0, 'severity'
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'System', 'pid': 4, 'severity': 'MEDIUM'}
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'Registry', 'pid': 232, 'severity': 'MEDIUM
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 336, 'severity': 'MED
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'smss.exe', 'pid': 780, 'severity': 'MEDIUM
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'msedgewebview2.exe', 'pid': 796, 'severity
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 920, 'severity': 'MED
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 1060, 'severity': 'ME
2026-03-20 22:06:34 [MEDIUM] {'type': 'New Process Detected', 'process': 'ModuleCoreService.exe', 'pid': 1136, 'seve
```

```
Command Prompt - python r
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.e
xe', 'pid': 8532, 'path': 'C:\\Windows\\System32\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'ShellHost.exe', 'pid
': 8928, 'path': 'C:\\Windows\\System32\\ShellHost.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CrossDeviceResume.ex
e', 'pid': 9272, 'path': 'C:\\Windows\\SystemApps\\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\\CrossDeviceResume.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'WidgetBoard.exe', 'p
id': 11156, 'path': 'C:\\Program Files\\WindowsApps\\MicrosoftWindows.Client.WebExperience_526.1202.40.0_x64__cw5n1h2txy
ewy\\WidgetBoard.exe'}
{'timestamp': '2026-03-20 22:14:25', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'SDXHelper.exe', 'pid
': 12552, 'path': 'C:\\Program Files\\Microsoft Office\\root\\Office16\\SDXHelper.exe'}

🔧 Auditing Windows services...

Total Services Found: 291

🚨 SERVICE ALERTS:

{'type': 'Service missing executable path', 'service': 'LSM', 'severity': 'MEDIUM'}
{'type': 'Service missing executable path', 'service': 'NetSetupSvc', 'severity': 'MEDIUM'}
{'type': 'Service running from suspicious location', 'service': 'BcastDVRUserService_c3c61', 'path': 'C:\\WINDOWS\\syste
m32\\svchost.exe -k BcastDVRUserService', 'severity': 'HIGH'}
📄 Security report generated: reports/final_report.txt

✔Monitoring cycle completed.

--------------------------------------------------------------------------------------------
Next scan in 60 seconds...
```

```
final_report.txt
File   Edit   View
WINDOWS SECURITY MONITORING REPORT
==========================================
Generated: 2026-03-20 22:15:44.023804

Total Running Processes: 295
Total Alerts: 322

Severity Breakdown:
  HIGH   : 3
  MEDIUM : 298
  LOW    : 21

Overall System Risk Level: HIGH

Detailed Alerts:
----------------------------------------
{'timestamp': '2026-03-20 22:15:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 7600}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'HIGH', 'type': 'Suspicious Parent-Child', 'parent': 'chrome.exe', 'child': 'cmd.exe', 'pid': 9948}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'MEDIUM', 'type': 'Suspicious Execution Path', 'process': 'CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.exe', 'path': 'C:
\\Users\\Vaishali\\AppData\\Local\\Temp\\vscode-stable-user-x64\\CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.exe', 'pid': 7700}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'MEDIUM', 'type': 'Suspicious Execution Path', 'process': 'CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.tmp', 'path': 'C:
\\Users\\Vaishali\\AppData\\Local\\Temp\\is-U7V1A.tmp\\CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.tmp', 'pid': 22792}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'MicrosoftStartFeedProvider.exe', 'pid': 3688, 'path': 'C:\\Program Files\\WindowsApps
\\Microsoft.StartExperiencesApp_1.241.0.0_x64__8wekyb3d8bbwe\\MicrosoftStartFeedProvider\\MicrosoftStartFeedProvider.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'smartscreen.exe', 'pid': 4908, 'path': 'C:\\Windows\\System32\\smartscreen.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 6348, 'path': 'C:\\Windows\\System32
\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.exe', 'pid': 7700, 'path': 'C:
\\Users\\Vaishali\\AppData\\Local\\Temp\\vscode-stable-user-x64\\CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 8532, 'path': 'C:\\Windows\\System32
\\backgroundTaskHost.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'ShellHost.exe', 'pid': 8928, 'path': 'C:\\Windows\\System32\\ShellHost.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'rundll32.exe', 'pid': 9052, 'path': 'C:\\Windows\\System32\\rundll32.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'CrossDeviceResume.exe', 'pid': 9272, 'path': 'C:\\Windows\\SystemApps
\\MicrosoftWindows.Client.CBS_cw5n1h2txyewy\\CrossDeviceResume.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'WidgetBoard.exe', 'pid': 11156, 'path': 'C:\\Program Files\\WindowsApps
\\MicrosoftWindows.Client.WebExperience_526.1202.40.0_x64__cw5n1h2txyewy\\WidgetBoard.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'SDXHelper.exe', 'pid': 12552, 'path': 'C:\\Program Files\\Microsoft Office\\root
\\Office16\\SDXHelper.exe'}
{'timestamp': '2026-03-20 22:15:34', 'severity': 'LOW', 'type': 'Unauthorized Process', 'process': 'backgroundTaskHost.exe', 'pid': 13688, 'path': 'C:\\Windows\\System32
```

Figure 8: Final Report.txt

final_report.txt

File Edit View


```
{'type': 'New Process Detected', 'process': 'chrome.exe', 'pid': 19192, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'chrome.exe', 'pid': 19216, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedgewebview2.exe', 'pid': 19228, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'chrome.exe', 'pid': 19260, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'Code.exe', 'pid': 19296, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'chrome.exe', 'pid': 19412, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'chrome.exe', 'pid': 19744, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'backgroundTaskHost.exe', 'pid': 19768, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'MfeBrowserHost.exe', 'pid': 19776, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'browserhost.exe', 'pid': 19912, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'ONENOTEM.EXE', 'pid': 19944, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'LinkedIn.exe', 'pid': 20304, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedgewebview2.exe', 'pid': 20364, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedge.exe', 'pid': 20544, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedgewebview2.exe', 'pid': 20816, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedge.exe', 'pid': 21008, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'msedgewebview2.exe', 'pid': 21160, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'pet.exe', 'pid': 21384, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'RuntimeBroker.exe', 'pid': 21468, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'RuntimeBroker.exe', 'pid': 21648, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'Code.exe', 'pid': 21888, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 22172, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 22228, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'aimgr.exe', 'pid': 22488, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'WmiPrvSE.exe', 'pid': 22512, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'Code.exe', 'pid': 22652, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'CodeSetup-stable-07ff9d6178ede9a1bd12ad3399074d726ebe6e43.tmp', 'pid': 22792, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'Clipchamp.Widgets.exe', 'pid': 23188, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'powershell.exe', 'pid': 23220, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'PhoneExperienceHost.exe', 'pid': 23368, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'conhost.exe', 'pid': 23448, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'conhost.exe', 'pid': 23452, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'backgroundTaskHost.exe', 'pid': 23532, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'Code.exe', 'pid': 23560, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'WindowsTerminal.exe', 'pid': 23768, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'ai.exe', 'pid': 23880, 'severity': 'MEDIUM'}
{'type': 'New Process Detected', 'process': 'svchost.exe', 'pid': 24204, 'severity': 'MEDIUM'}
{'type': 'Service missing executable path', 'service': 'LSM', 'severity': 'MEDIUM'}
{'type': 'Service missing executable path', 'service': 'NetSetupSvc', 'severity': 'MEDIUM'}
{'type': 'Service running from suspicious location', 'service': 'BcastDVRUserService_c3c61', 'path': 'C:\\WINDOWS\\system32\\svchost.exe -k BcastDVRUserService', 'severity': 'HIGH'}
```


Ln 315, Col 102 35,061 characters Plain text 100% Windows (CRLF) UTF-8

10. DETECTION TECHNIQUES IMPLEMENTED

The **Windows Service & Process Monitoring Agent** uses multiple detection techniques to identify suspicious system behavior. These techniques are based on behavioral analysis and rule-based monitoring to detect anomalies in processes and services.

1. Parent–Child Process Monitoring
This technique analyzes the relationship between processes and their parent processes. In normal system behavior, processes usually follow predictable execution chains. The monitoring agent checks for abnormal parent–child relationships, such as a document editor launching a command shell or scripting tool. Such behavior may indicate malicious macros, exploitation attempts, or unauthorized script execution.

2. Execution Path Monitoring
The system examines the file path from which a process is executed. Malware often runs from temporary folders or user-writable directories such as *Temp*, *Downloads*, or *AppData*. If a process is detected running from these locations, the monitoring agent flags it as potentially suspicious.

3. Baseline Anomaly Detection
A baseline of trusted processes is generated during the initial setup of the system. During monitoring cycles, the agent compares running processes against this baseline. If a process is detected that is not included in the trusted list, it may be flagged as an unknown or unauthorized process.

4. Service Configuration Analysis
The monitoring agent audits Windows services to analyze their configuration details such as service name, startup type, and executable path. Services running from unusual directories or newly registered services with unfamiliar names may indicate malware persistence mechanisms.

5. Rule-Based Detection
The system uses predefined detection rules to identify suspicious behavior patterns. These rules analyze process relationships, execution paths, and service configurations to determine whether an activity should generate a security alert.

11. TESTING METHODOLOGY

To evaluate the effectiveness of the **Windows Service & Process Monitoring Agent**, several controlled test scenarios were performed. The purpose of these tests was to verify whether the monitoring system could successfully detect suspicious processes, abnormal execution patterns, and unusual service configurations.

Test Scenario 1 – Suspicious Parent–Child Process Execution
In this test, command-line tools such as **PowerShell or Command Prompt** were launched from other applications. The monitoring agent analyzed the parent–child process relationship and generated alerts when suspicious execution chains were detected.

Test Scenario 2 – Executable Running from Temporary Directory
A test executable was launched from a temporary folder such as the *Temp* or *Downloads* directory. Since attackers often run malware from user-writable directories, the monitoring system flagged the process as suspicious based on its execution path.

Test Scenario 3 – Unauthorized Process Detection
A new process that was not included in the trusted baseline list was executed. The monitoring agent compared the running process against the baseline and generated an alert for an unknown or unauthorized process.

Test Scenario 4 – Suspicious Service Configuration
A service with an unusual executable path or unfamiliar name was monitored. The service auditing module inspected the configuration and flagged services that appeared abnormal or potentially malicious.

Testing Results
During these tests, the monitoring agent successfully detected suspicious behaviors and generated alerts with appropriate severity levels. The results confirmed that the implemented detection rules were able to identify abnormal system activity and provide useful security insights.

12. SAMPLE OUTPUTS / RESULTS

During the testing phase of the **Windows Service & Process Monitoring Agent**, the system successfully generated alerts when suspicious activities were detected. The alerts provide information about abnormal process behavior, unauthorized processes, and suspicious service configurations.

Example Alert Messages

• **Suspicious Process Detected:** winword.exe → powershell.exe
This alert indicates that Microsoft Word launched a PowerShell process. Such behavior may suggest the presence of malicious macros or scripts executing through the document.

• **Unauthorized Process Detected:** unknown.exe
This alert appears when a process that is not present in the trusted baseline or whitelist is detected running on the system.

• **Suspicious Service Detected:** UnknownServiceXYZ
This alert is generated when a new or unfamiliar Windows service is detected, which may indicate a persistence mechanism used by malware.

Example Monitoring Report Summary

```
WINDOWS SECURITY MONITORING REPORT
----------------------------------------
Total Running Processes: 120
Total Alerts Generated: 5

Severity Breakdown:
HIGH   : 2
MEDIUM : 2
LOW    : 1
```

The monitoring report provides a summary of system activity, the number of alerts detected, and their severity levels. This structured output helps users quickly understand the overall security status of the system and identify potential threats that require further investigation.

13. OBSERVATIONS / ANALYSIS

During the testing and evaluation of the **Windows Service & Process Monitoring Agent**, several important observations were made regarding system behavior and detection accuracy.

1. Windows Processes Follow Predictable Parent–Child Relationships
It was observed that most legitimate Windows processes follow consistent and predictable parent–child relationships. For example, system processes are usually started by trusted parent processes such as system services or the Windows operating system itself. Because of this predictable structure, any unusual parent–child process chain can be considered suspicious and may indicate malicious activity.

2. Suspicious Behavior is Noticeable Through Abnormal Execution Chains
When a process is launched from an unexpected parent process, it becomes easier to detect abnormal behavior. For example, if a document editing application launches a command shell or scripting tool, it may indicate malicious macros or exploitation. This observation highlights how analyzing execution chains helps identify potential threats.

3. Baseline Comparison Reduces False Positives
Initially, many normal processes were flagged as suspicious. After implementing a **baseline comparison mechanism**, the monitoring system was able to distinguish between trusted processes and unknown ones. This significantly reduced the number of false alerts and improved the reliability of the monitoring system.

4. Service Auditing Helps Detect Persistence Mechanisms
Monitoring Windows services revealed that services can be used as a persistence mechanism by attackers. Suspicious services often run from unusual directories or have unknown executable paths. By auditing service configurations, the system can identify such indicators of compromise.

14. ADVANTAGES OF THE SYSTEM

The **Windows Service & Process Monitoring Agent** offers several advantages that make it useful for monitoring system activity and detecting potential security threats.

1. Real-Time Monitoring of Processes and Services
The system continuously monitors running processes and Windows services. This allows the tool to detect suspicious behavior as soon as it occurs, helping in the early identification of potential threats or unauthorized activities on the system.

2. Detection Using Rule-Based Analysis
The monitoring agent uses predefined detection rules to analyze system behavior. By checking parent–child process relationships, execution paths, and service configurations, the system can identify abnormal patterns that may indicate malicious activity.

3. Lightweight and Modular Design
The tool is designed using a modular architecture where different scripts handle specific tasks such as process monitoring, detection, service auditing, and report generation. This makes the system lightweight, easy to maintain, and simple to extend with additional features.

4. Generates Structured Security Reports
The monitoring agent produces well-organized reports that summarize detected alerts, system activity, and severity levels. These reports help users easily understand the security status of the system and support investigation of suspicious events.

5. Helps Identify Unauthorized System Activity
By comparing running processes against a trusted baseline, the system can identify unknown or unauthorized processes. This helps detect potentially malicious programs that should not normally be running on the system.

15. LIMITATIONS / DISADVANTAGES

Although the **Windows Service & Process Monitoring Agent** is useful for detecting suspicious system behavior, it has several limitations that affect its detection capabilities.

1. Cannot Detect Kernel-Level Malware
The monitoring agent operates in **user mode**, which means it monitors processes and services that run at the application level. However, some advanced malware operates at the **kernel level**, which has deeper access to the operating system. Since the tool does not interact with the kernel, such malware may bypass the monitoring system and remain undetected.

2. No Network Traffic Analysis
The current system focuses mainly on **process and service monitoring**. It does not inspect network activity such as outgoing connections, unusual ports, or communication with external servers. As a result, threats involving **data exfiltration or command-and-control communication** cannot be detected by the tool.

3. Rule-Based Detection Approach
The monitoring agent uses **predefined detection rules** to identify suspicious behavior. While these rules can detect many known attack patterns, they may not identify new or sophisticated techniques that do not match existing rules. This means the system may miss **previously unseen attack methods**.

4. Limited Detection of Fileless Malware
Some modern attacks use **fileless malware**, which runs directly in memory without creating files on the system. Because the monitoring agent mainly checks process names, execution paths, and services, such attacks may be more difficult to detect.

16. FUTURE SCOPE

Although the **Windows Service & Process Monitoring Agent** successfully performs basic monitoring and detection, there are several improvements that can be implemented in the future to make the system more advanced and closer to enterprise-level security tools.

One possible improvement is the addition of **registry monitoring**. Many malware programs create or modify registry keys to maintain persistence on a system. By monitoring important registry locations such as startup entries, the system could detect unauthorized changes that indicate potential malicious activity.

Another enhancement would be **network activity monitoring**. Currently, the tool focuses mainly on processes and services. By analyzing network connections, the system could detect suspicious outbound communications, unusual ports, or unknown external connections that may indicate data exfiltration or command-and-control communication.

The system could also include **automated response mechanisms**. Instead of only generating alerts, the monitoring agent could automatically terminate suspicious processes or disable potentially malicious services to prevent further damage.

Adding **digital signature verification** would also improve detection accuracy. The system could check whether executables are signed by trusted vendors, helping to distinguish legitimate software from potentially malicious files.

Future versions of the project could also incorporate **machine learning-based anomaly detection** to identify unusual system behavior automatically without relying only on predefined rules.

Finally, the tool could be enhanced with **SIEM (Security Information and Event Management) integration** to send logs and alerts to a centralized security monitoring platform, enabling security analysts to monitor multiple systems from a single dashboard.

These improvements would make the monitoring agent more powerful and bring it closer to real-world **enterprise endpoint detection and response (EDR) solutions**.

17. LEARNING OUTCOMES

The development of the **Windows Service & Process Monitoring Agent** provided valuable practical learning and helped strengthen several technical and analytical skills related to cybersecurity and system monitoring.

Through this project, I gained a deeper **understanding of Windows process architecture**, including how processes are created, how parent–child relationships work, and how different applications interact within the operating system. This knowledge is essential for identifying abnormal execution patterns that may indicate malicious activity.

The project also helped me learn how to **implement behavioral detection logic**. Instead of relying only on predefined malware signatures, the system analyzes process behavior, execution paths, and service configurations to detect suspicious activity. This approach reflects modern cybersecurity practices used in Security Operations Centers (SOC).

Another important outcome was gaining experience in **developing monitoring tools using Python**. I worked with libraries such as *psutil* and *WMI* to collect system information and build scripts capable of analyzing processes and services in real time.

Additionally, the project improved my ability to **analyze system activity for threat detection**. By studying how attackers misuse legitimate Windows processes and services, I learned how monitoring tools can detect potential persistence mechanisms, unauthorized executions, and abnormal system behavior.

Finally, I developed skills in **designing structured security reports and alert logs**, which are essential for documenting findings and presenting monitoring results clearly during security investigations.

18. CONCLUSION

The **Windows Service & Process Monitoring Agent** project successfully demonstrates how endpoint monitoring can be used to detect suspicious activity within a Windows system. The tool continuously monitors running processes and services, analyzes parent–child process relationships, and checks execution paths to identify abnormal behavior that may indicate potential security threats.

Through the implementation of rule-based detection and baseline comparison, the system is able to highlight unauthorized processes, suspicious execution chains, and unusual service configurations. These capabilities simulate the type of monitoring techniques commonly used in Security Operations Centers (SOC) to detect malware activity, persistence mechanisms, and system compromise.

During the development of this project, several challenges such as permission issues, incomplete process data, and false positives were encountered. Solving these problems improved the stability and reliability of the monitoring agent while also strengthening practical troubleshooting and debugging skills.

Overall, this project provided valuable hands-on experience with **Windows internals, process monitoring, and defensive cybersecurity techniques**. It enhanced my understanding of how system behavior can be analyzed to detect threats and helped develop practical skills in **security monitoring, detection logic, and Python-based system analysis**, making it an important step toward building a career in cybersecurity.

19. PROJECT DELIVERABLES

The completed project includes the following deliverables:

1. **Project Documentation Report** – A comprehensive report detailing the project objectives, system architecture, implementation, testing, and analysis.
2. **Python Monitoring Scripts** – All source code files required for process and service monitoring, including main.py, detector.py, process_monitor.py, service_audit.py, baseline_generator.py, and report_generator.py.
3. **Sample Logs and Alert Outputs** – Example outputs generated during testing, showing detected suspicious processes, unauthorized activity, and service anomalies.
4. **System Architecture Diagrams** – Flowcharts and module diagrams illustrating the monitoring workflow and detection logic.
5. **Final Presentation Slides** – A PowerPoint presentation summarizing the project, workflow, results, and key findings for demonstration purposes.

20. REFERENCES / BIBLIOGRAPHY

- **Microsoft Windows Services Documentation** – Provides detailed information on Windows service management and APIs.
  https://docs.microsoft.com/en-us/windows/win32/services/
- **Psutil Python Documentation** – Documentation for the psutil library used for process monitoring and system resource information.
  https://psutil.readthedocs.io/
- **WMI Python Documentation** – Documentation for the WMI Python library used to interact with Windows Management Instrumentation.
  https://pypi.org/project/WMI/
- **Python Official Documentation** – Reference for Python programming language and standard library functions.
  https://docs.python.org/
- **Draw.io Diagram Tool** – Tool for creating system architecture and workflow diagrams.
  https://app.diagrams.net/